

AI ASSI CODING-13.4

HTNO:2303A510B4

Batch:14

Task 1: Refactoring Data Transformation

Logic

Scenario

You are maintaining a data preprocessing script where numerical

transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

```
values = [2, 4, 6, 8, 10]
```

```
doubled = []  
  
for v in values:  
  
    doubled.append(v * 2)  
  
print(doubled)
```

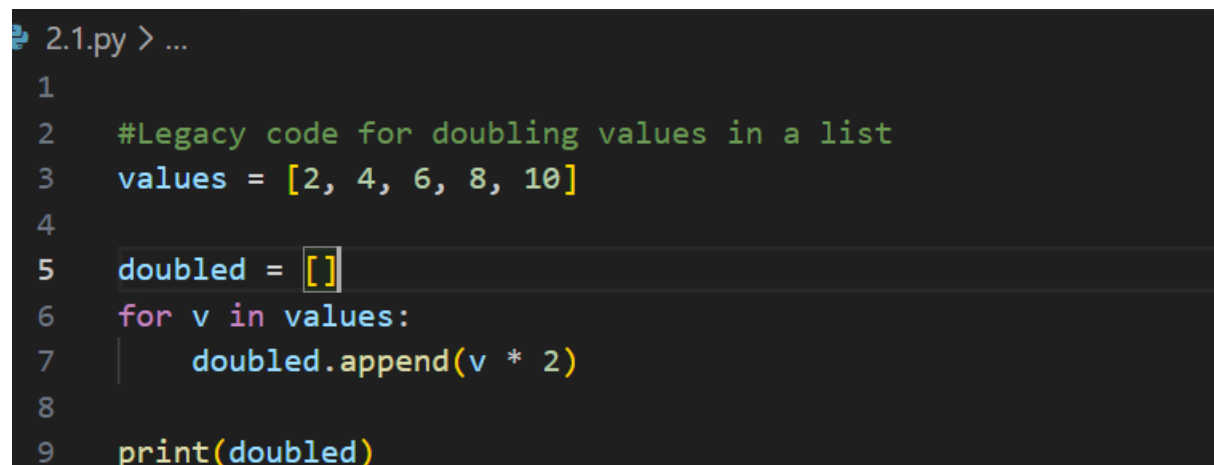
Expected Output

[4, 8, 12, 16, 20]

Prompt:

#Legacy code for doubling values in a list

Code:

A screenshot of a Python code editor with a dark background. The code is as follows:

```
2.1.py > ...  
1  
2     #Legacy code for doubling values in a list  
3     values = [2, 4, 6, 8, 10]  
4  
5     doubled = []  
6     for v in values:  
7         doubled.append(v * 2)  
8  
9     print(doubled)
```

#Refactor the code using list comprehension

Code:

```
10 #Refactor the code using list comprehension
11 values = [2, 4, 6, 8, 10]
12 doubled = [v * 2 for v in values]
13 print(doubled)
14
```

Output:

```
Drive/Documents/pthon learning/2.1.py"
[4, 8, 12, 16, 20]
[4, 8, 12, 16, 20]
PS C:\Users\nithe\OneDrive\Documents\pthon learning>
```

Justification:

- 1.Using a list comprehension simplifies the loop logic and produces the transformed list **in** a single readable line.
- 2.It follows Pythonic coding practices, making the code shorter, cleaner, and easier to maintain while keeping the same output.

Task 2: Improving Text Processing Code

Readability

Scenario

You are working on a log message generator that builds messages word

by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves",  
"quality"]
```

```
message = ""
```

```
for w in words:
```

```
    message += w + " "
```

```
print(message.strip())
```

Expected Output

Refactoring with AI improves quality

Prompt:

#Legacy code for concatenating words into a message

Code:

```
1  #Legacy code for concatenating words into a message
2  words = ["Refactoring", "with", "AI", "improves", "quality"]
3
4  message = ""
5  for w in words:
6      message += w + " "
7
8  print(message.strip())
```

#Refactor the code and more efficient and readable approach

```
9  #Refactor the code and more efficient and readable approach
10 words = ["Refactoring", "with", "AI", "improves", "quality"]
11 message = " ".join(words)
12 print(message)
```

Output:

```
PS C:\Users\nithe\OneDrive\Desktop> python3 ai_exam.py
Refactoring with AI improves quality
Refactoring with AI improves quality
PS C:\Users\nithe\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

- 1.It is **more efficient than repeated string concatenation**, which creates many temporary strings inside the loop.
- 2.This approach follows **Pythonic coding practices**, making the code shorter, cleaner, and easier to maintain.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Expected Output

Default timeout used

Prmpt:

#Legacy code for checking for a key in a dictionary values

Code:

```
1  #Legacy code for checking for a key in a dictionary values
2  config = {"host": "localhost", "port": 8080}
3
4  if "timeout" in config:
5      print(config["timeout"])
6  else:
7      print("Default timeout used")
```

#Refactor the code using safer dictionary access methods and the behavior remains the same for missing keys.

```
8  #Refactor the code using safer dictionary access methods and the behavior remains the same for missing keys.
9  config = {"host": "localhost", "port": 8080}
10 timeout = config.get("timeout", "Default timeout used")
11 print(timeout)
```

Output:

```
Default timeout used  
Default timeout used  
PS C:\Users\nithe\AppData\Local\Programs\Microsoft VS Code> █
```

Justification:

- 1.The **dict.get()** method **safely retrieves dictionary values** without raising an error if the key does not exist.
2. It **simplifies the code by removing manual key checking**, making the program shorter and more readable

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"
```

```
x, y = 10, 2
```

```
if action == "add":
```

```
    result = x + y
```

```
elif action == "subtract":
```

```
    result = x - y
```

```
elif action == "multiply":
```

```
    result = x * y
```

```
elif action == "divide":
```

```
result = x / y
```

```
else:
```

```
result = None
```

```
print(result)
```

Expected Output

5.0

Prompt:

#Legacy code multiple if-else conditions for operations

Code:

```

1  #Legacy code multiple if-else conditions for operations
2  action = "divide"
3  x, y = 10, 2
4
5  if action == "add":
6      result = x + y
7  elif action == "subtract":
8      result = x - y
9  elif action == "multiply":
10     result = x * y
11  elif action == "divide":
12     result = x / y
13  else:
14     result = None
15
16  print(result)

```

#Refactor the logic using mapping techniques while preserving functionality cleaner and scalable

```

18  #Refactor the logic using mapping techniques while preserving functionality cleaner and scalable
19  action = "divide"
20  x, y = 10, 2
21
22  operations = {
23      "add": x + y,
24      "subtract": x - y,
25      "multiply": x * y,
26      "divide": x / y
27  }
28
29  result = operations.get(action, None)
30
31  print(result)

```

Justification:

1.Dictionary mapping **reduces multiple if-elif statements**, making the code shorter and more readable.

2.It **improves scalability**, because new operations can be added easily without changing the existing logic.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.

- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser",  
"marker"]
```

```
found = False
```

```
for item in inventory:
```

```
    if item == "eraser":
```

```
        found = True
```

```
    break
```

```
print("Item Available" if found else "Item Not  
Available")
```

Expected Output

Item Available

Legacy Code:

```

1  #Legacy code
2  inventory = ["pen", "notebook", "eraser", "marker"]
3
4  found = False
5  for item in inventory:
6      if item == "eraser":
7          found = True
8          break
9
10 print("Item Available" if found else "Item Not Available")

```

#refactor the logic into a more concise and readable form.

```

12 #refactor the logic into a more concise and readable form.
13 inventory = ["pen", "notebook", "eraser", "marker"]
14
15 print("Item Available" if "eraser" in inventory else "Item Not Available")

```

Output:

```

C:\Users\nithe\OneDrive\Desktop>python 1.py
Item Available
Item Available
PS C:\Users\nithe\AppData\Local\Programs\Microsoft VS Code>

```

Justification:

1.The in operator directly checks if an element exists in the collection, eliminating the need for a manual loop.

2.This makes the code shorter, clearer, and easier to maintain while keeping the same functionality.